The Rules Are 7% of the Program

Josh · Secure Sovereign

Published October 30, 2026 · Updated October 31, 2026

A line count of Bitcoin Core 32.1

*Release **32.1**, branch `32.x`, commit `f0de87f`, 26 October 2026. Current-tree figures are tokei 14 code lines. History figures are git numstat on non-merge commits.*

The consensus rules are **10,122** code lines. The implementation they sit in is **151,004**. The code that applies those rules, stores the result, or chooses what to relay and mine is **20,366**, twice the rules. The other **120,516** lines are the wallet, the GUI, RPC, P2P transport, and the rest of the node.

A wrong rule forks the network. A policy disagreement does not. The two live in the same program, and the rules are the thin slice. That is the measurement behind *The Vertical Layer Problem* and *Why Bitcoin Needs a Specification That Humans Can Read*.

Contents

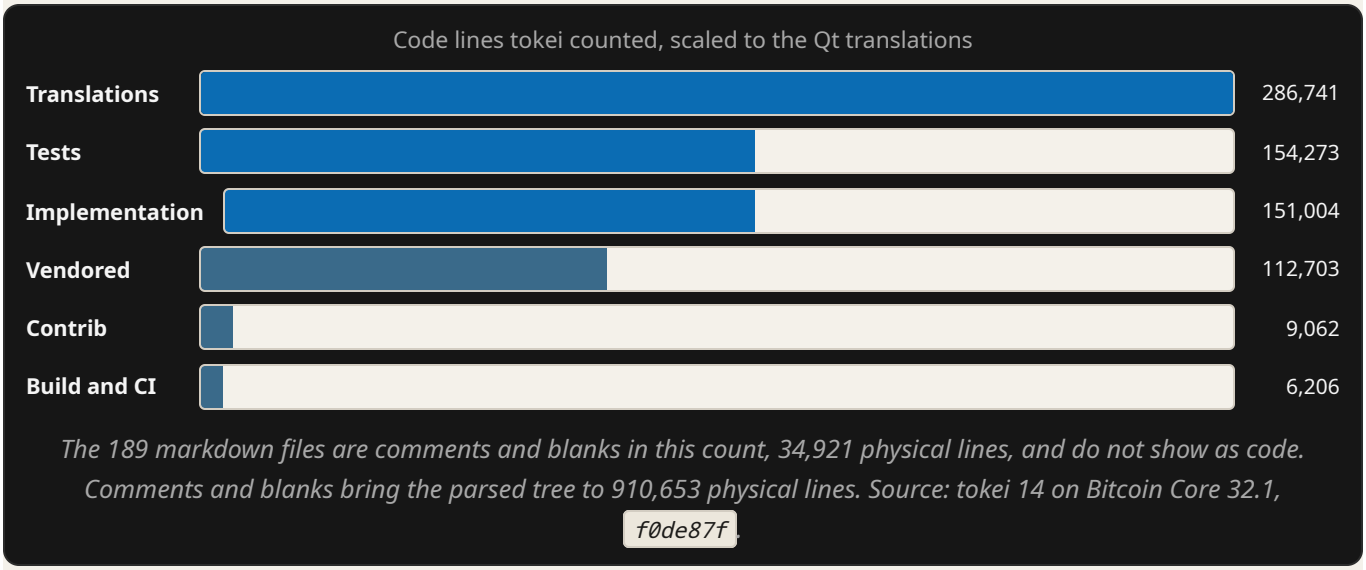
- I. What was counted
- II. The tree
- III. The implementation
- IV. The rules
- V. What sits beside the rules
- VI. Tests
- VII. Bundled code
- VIII. Who added the lines
- Where the work went
- What changes in the same commit
- IX. Who last touched what remains
- Who still edits a system
- The size of the contract
- X. Patch debt
- XI. What the count is for
- XII. Method

I. What was counted

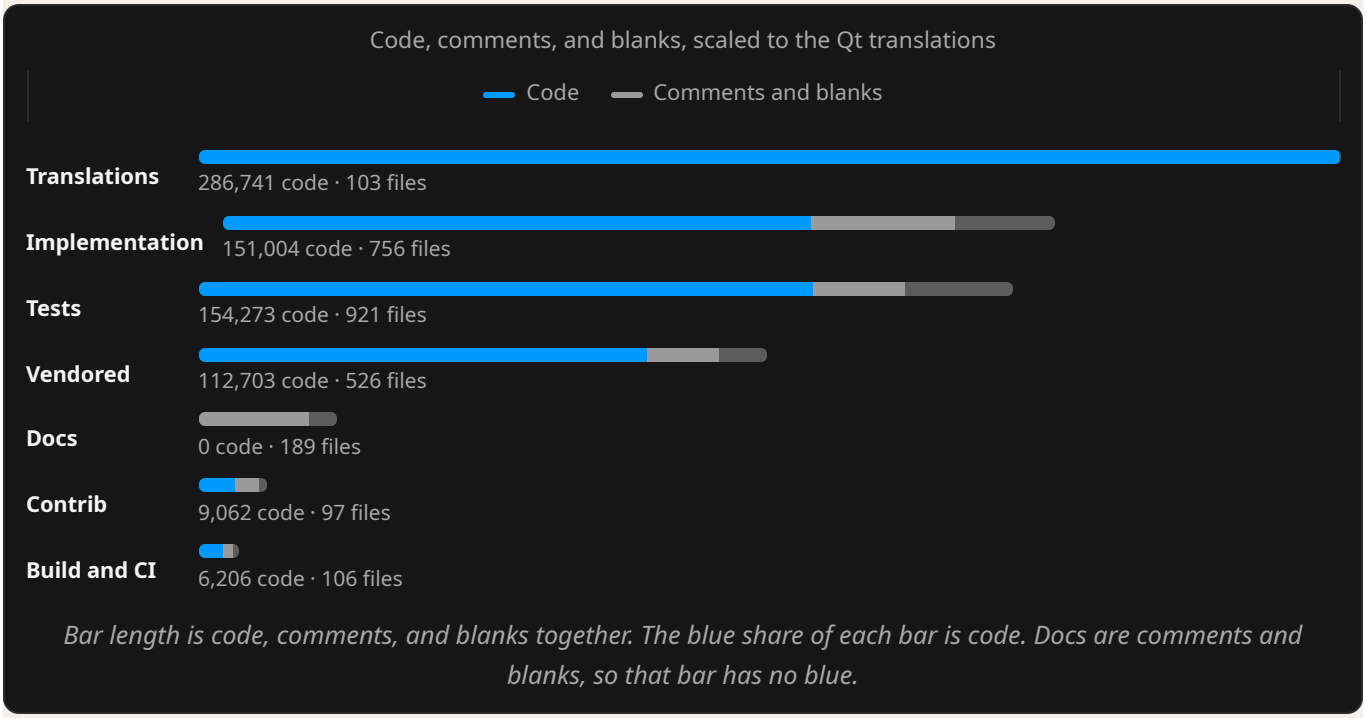
Tokei parsed **2,698** files and **719,989** code lines. Every one of those files is in exactly one bucket below. Git tracks 3,020 files. The 322 tokei skipped are 215 text files (31,095 physical lines, mostly Qt forms, CI glue, and licenses) and 107 binary files (4.82 MB). They are not in the code totals.

The headline is the implementation, not the 719,989. Qt locale files are 286,741 of those lines and are translations, counted by tokei as TypeScript. Vendored libraries are another 112,703. First-party code, implementation plus tests plus contrib, build, and docs, is **320,545**.

II. The tree



The 189 markdown files are comments and blanks in this count, 34,921 physical lines, and do not show as code. Comments and blanks bring the parsed tree to 910,653 physical lines. Source: tokei 14 on Bitcoin Core 32.1, `f0de87f`.



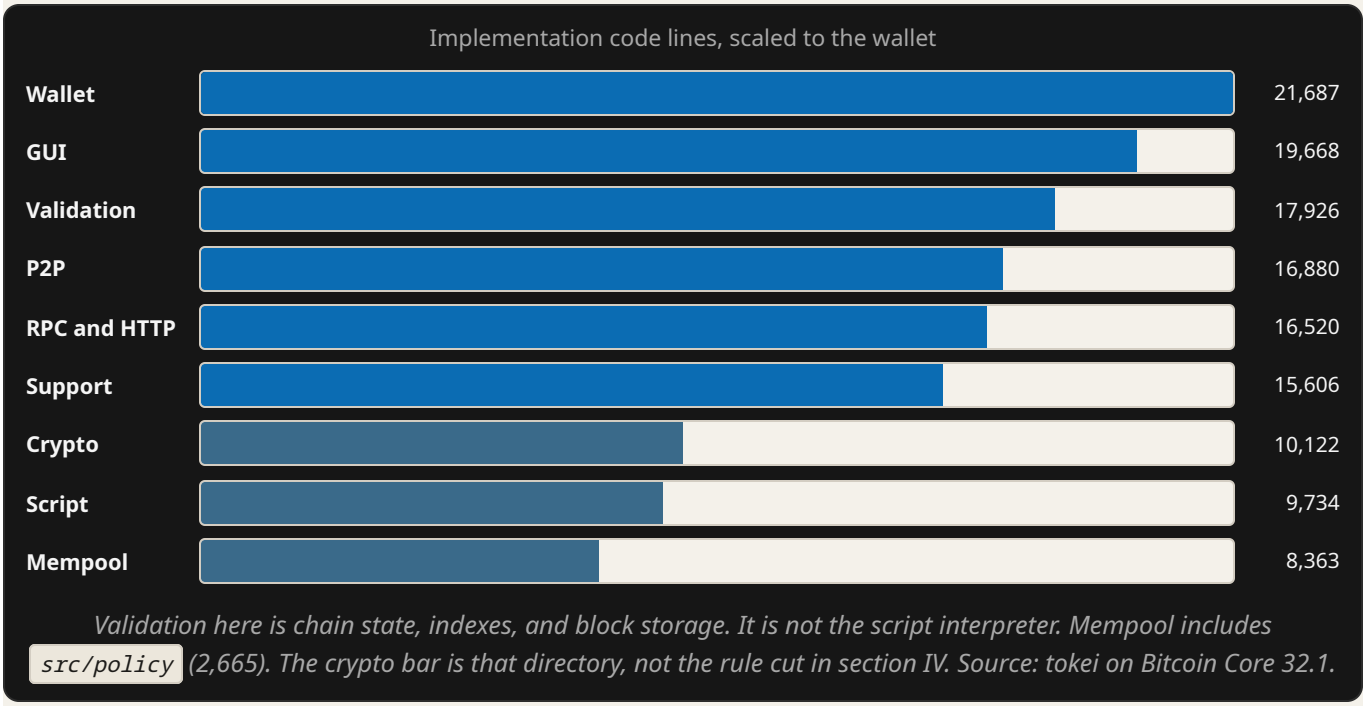
Bar length is code, comments, and blanks together. The blue share of each bar is code. Docs are comments and blanks, so that bar has no blue.

Tests are 50.5% of implementation-plus-tests. Drop the JSON, hex, and HTML fixtures and test code is 141,369, which is 48.4% of implementation plus that test code. The program carries about as much test code as implementation code. On this release the test pile is the larger of the two.

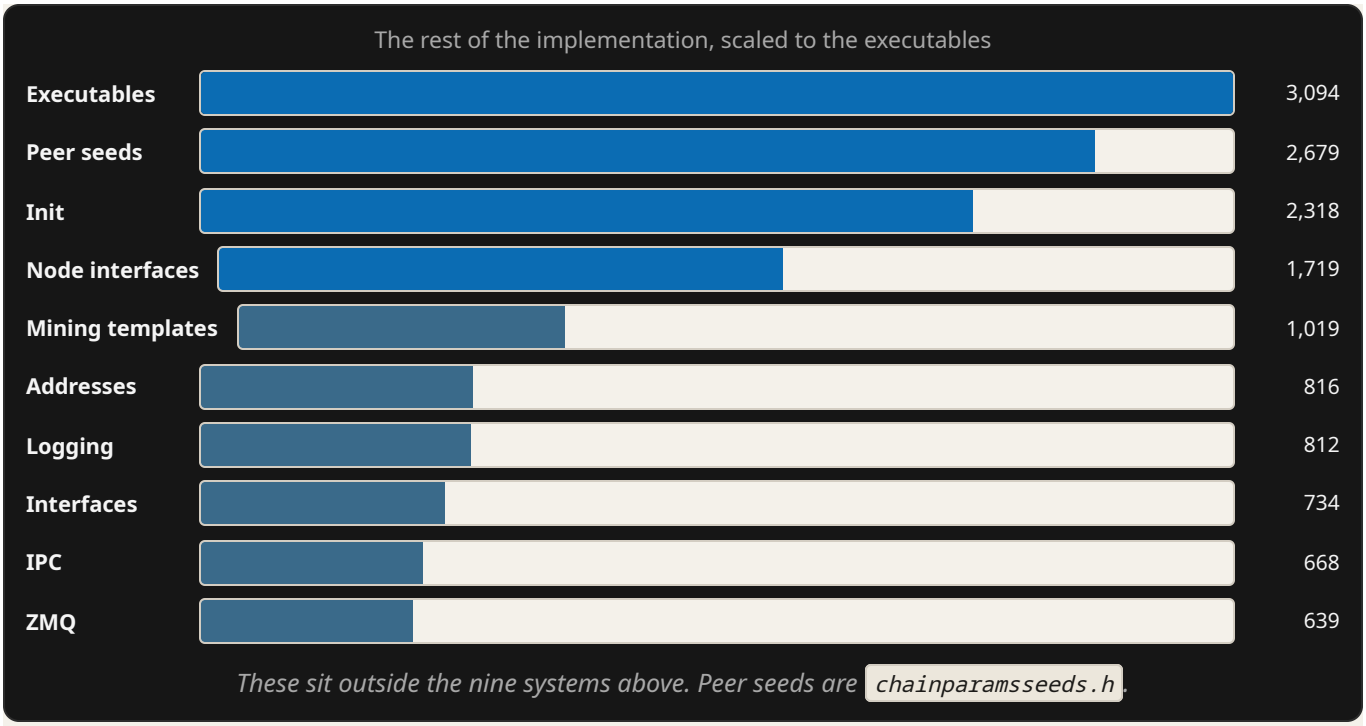
First-party languages, the 320,545: C++ 181,104, Python 69,946, C and C++ headers 43,805, JSON 13,003, build files 10,667, Rust 1,141, other 879. Implementation bodies are 73.8% C++ (111,494) and 25.6% headers (38,669). The other 841 lines are neither.

III. The implementation

Seven hundred fifty-six files. The median file is 66 code lines. Thirty-three files of 1,000 lines or more hold 38.9% of the implementation. Fifty-four files hold half of it.



Validation here is chain state, indexes, and block storage. It is not the script interpreter. Mempool includes `src/policy` *(2,665). The crypto bar is that directory, not the rule cut in section IV. Source: tokei on Bitcoin Core 32.1.*

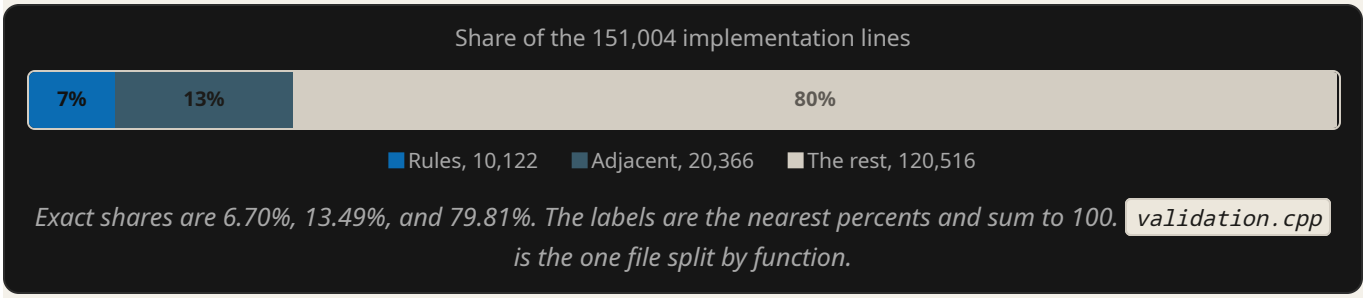


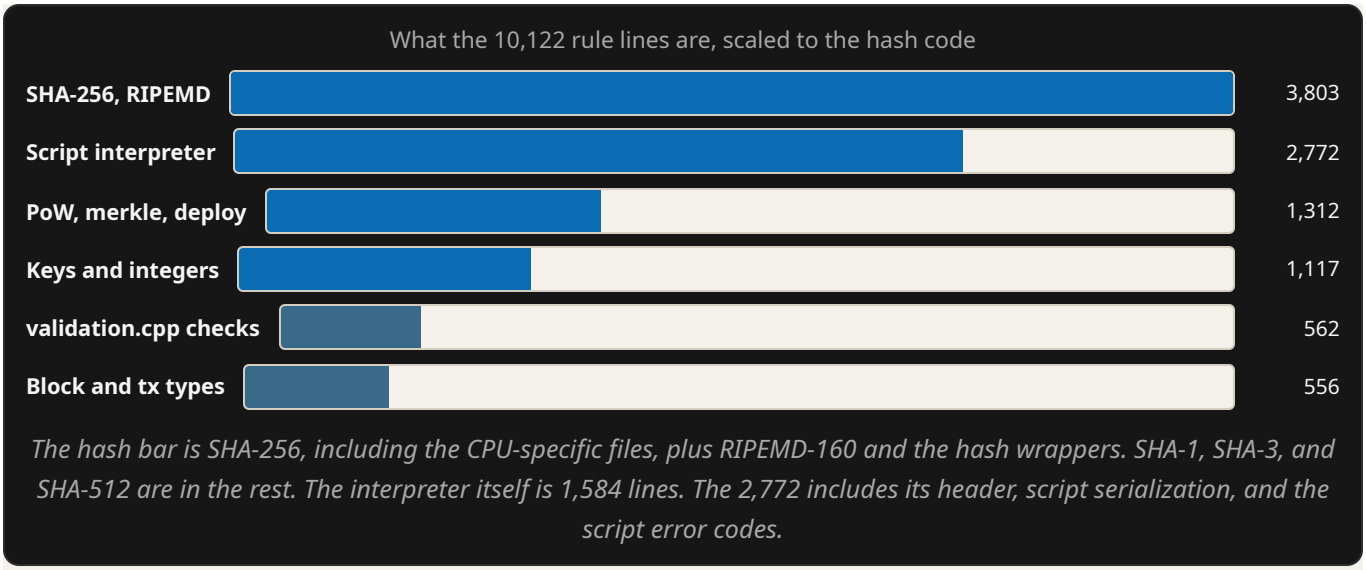
The largest files are `validation.cpp` (4,247), `net_processing.cpp` (4,220), `wallet.cpp` (3,488), `net.cpp` (3,248), `rpc/blockchain.cpp` (3,172), `chainparamsseeds.h` (2,679), `script/descriptor.cpp` (2,534), and `txgraph.cpp` (2,359).

`src/node` is not all P2P. Block storage and chainstate in that directory are in the validation bar. P2P is the network files only.

IV. The rules

Consensus here means the code that decides whether a block or a transaction is valid. The bundled signature library is outside this split. It is counted in section VII.





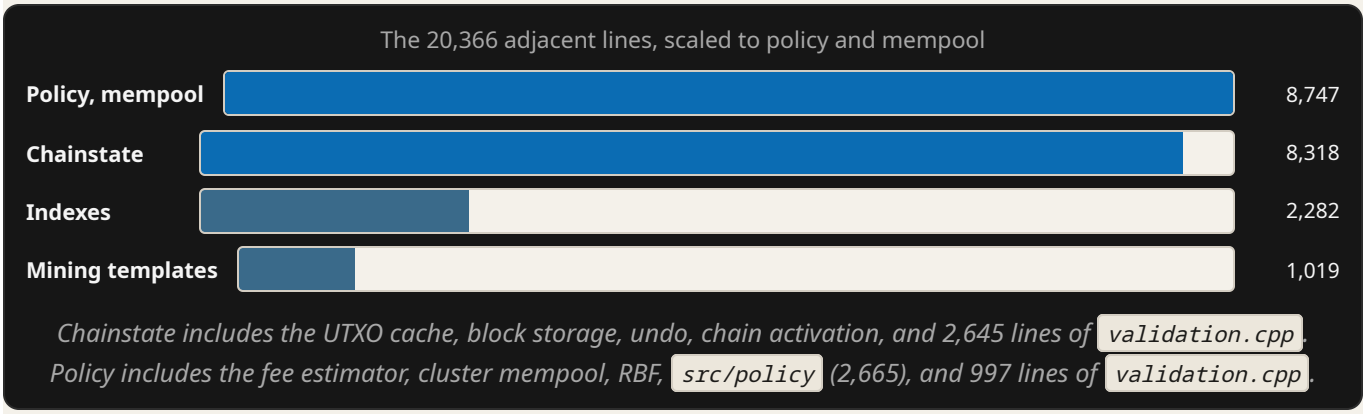
Core's exported library, `libbitcoinconsensus`, is **4,757** of these rule lines: the interpreter, script, merkle, transaction checks, hashes, and the basic types. The other **5,365** rule lines are outside it. **3,568** of those are the SHA-256 and RIPEMD-160 implementations. The other **1,797** are proof of work, versionbits, deployments, signet, transaction input checks, the 562 lines in `validation.cpp`, and a handful of headers.

The kernel library is a different list. Its own comment says the list will shrink. The named sources still compile policy and mempool, so the kernel target is not this boundary. `bitcoin_crypto` sums to 5,605 and includes `cleanse.cpp` and `lockedpool.cpp`.

`src/script/solver.h` says the solver is used by policy and the wallet, and not by consensus. Descriptors and miniscript are in the 120,516.

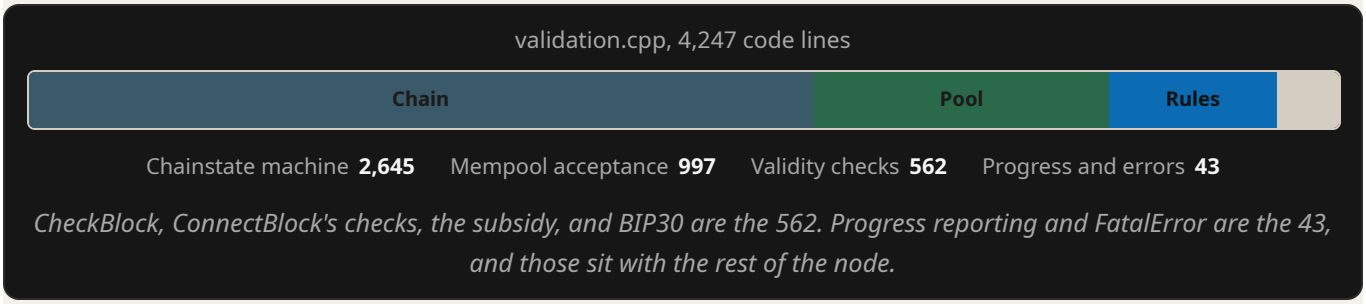
V. What sits beside the rules

Adjacent means the code that applies the rules, stores the result, or chooses what to relay and mine. An index can be wrong without making a block invalid. A policy limit can change without forking.



validation.cpp is mostly the machine around the checks

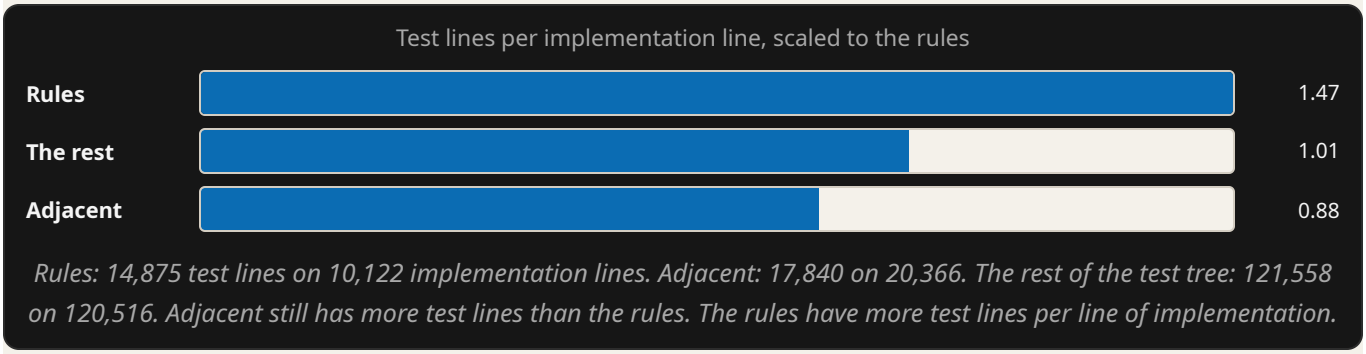
The file is 4,247 code lines. The split is at function boundaries, comment-stripped, then scaled so the parts still sum to tokei's figure. ConnectBlock's checks stay with the rules. The undo write at the end of that function stays with chainstate. MemPoolAccept stays with policy.

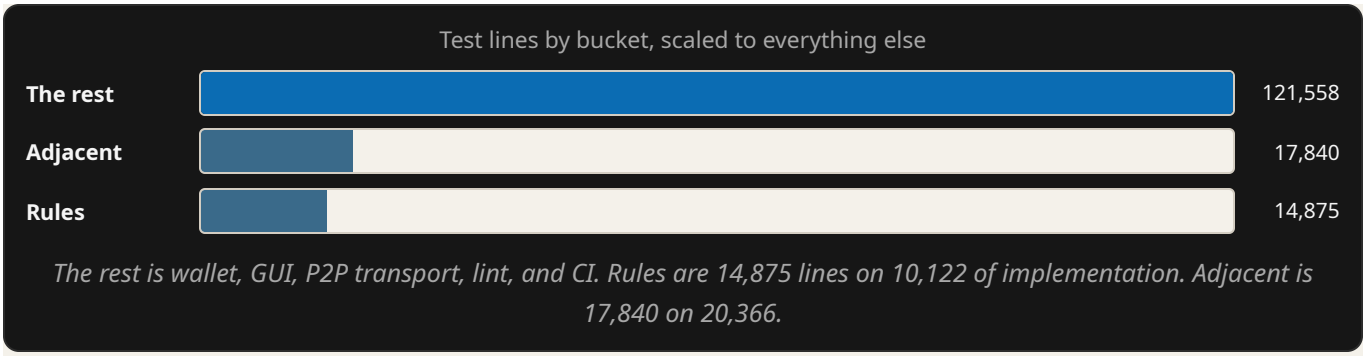


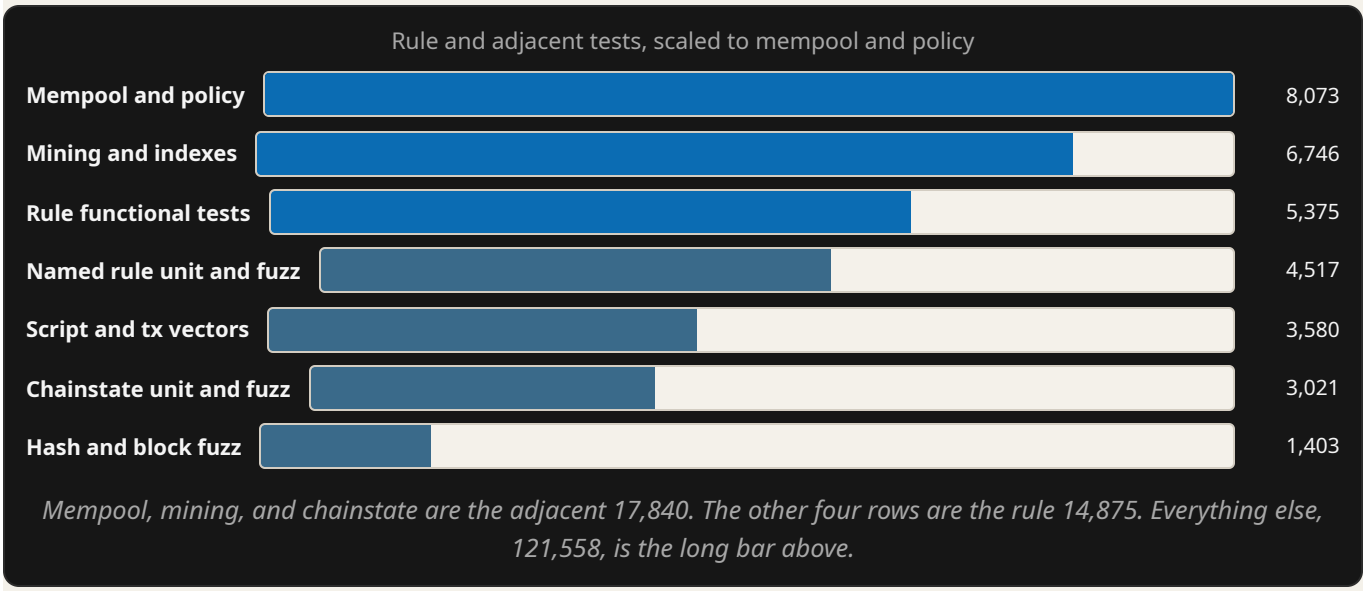
VI. Tests

Tests are 154,273 code lines: functional Python 62,437, unit 42,026, fuzz 22,254, fixtures 12,904, bench 6,799, wallet 3,193, lint 1,774, CI 1,135, GUI 1,133, harness 410, IPC 208.

A filename cut that stops at unit tests understates the functional suite. The assignment below puts every test file in one bucket. Rule tests are the named unit and fuzz files, further unit and fuzz of the rule types, the script and transaction vectors, and the functional tests of validity rules, including `p2p_segwit.py` (1,391), `feature_taproot.py` (1,353), and `feature_block.py` (927). Adjacent tests keep the mempool and policy names and add mining, assumeutxo, pruning, indexes, fee estimation, and package RPC. Benchmarks that share those names stay out of the ratio.







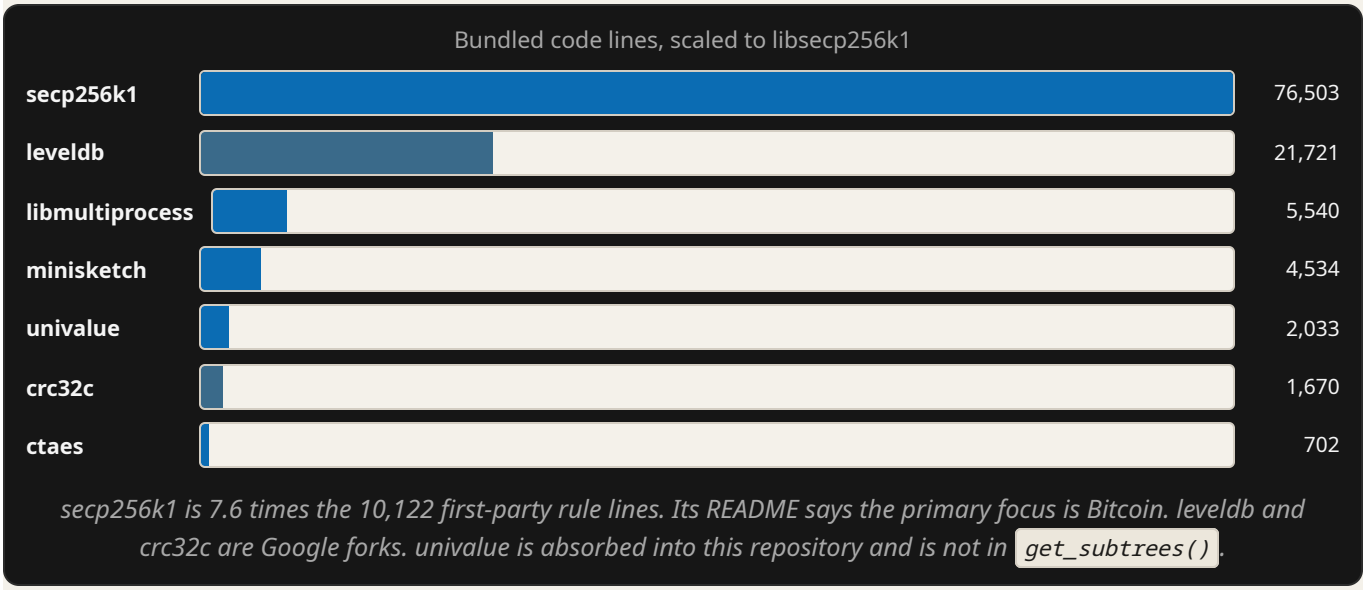
The 3,580 vector lines are `script_tests.json` (2,614), `tx_valid.json` (426), `tx_invalid.json` (313), and `invalid_txs.py` (227). The rest row includes wallet, GUI, P2P transport, lint, and CI. A test that calls the interpreter under a name this list does not use is in that row.

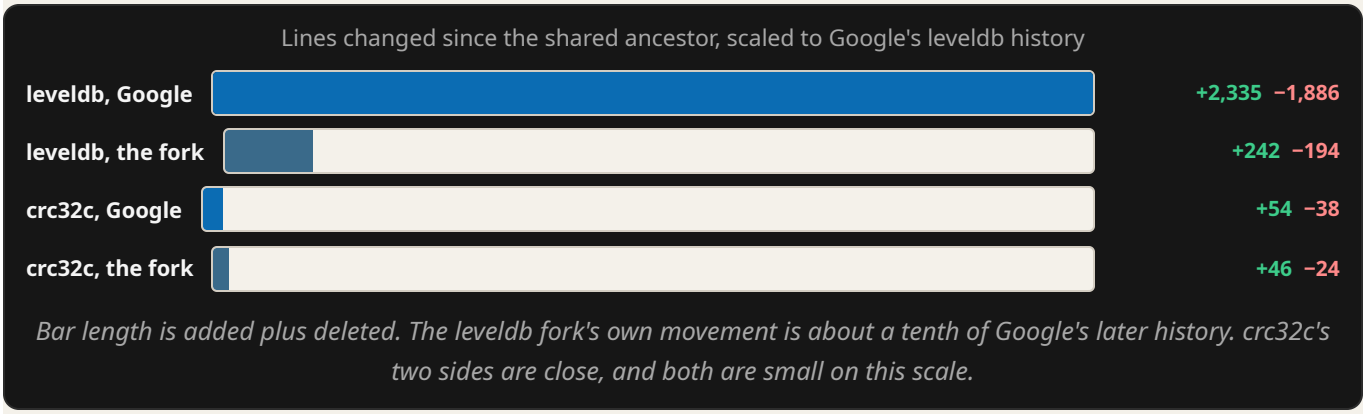
VII. Bundled code

Vendored trees sum to 112,703 code lines. **89,312** (79%) are maintained as Bitcoin Core projects. **23,391** (21%) are forks of Google libraries.

The fork's own patch is small next to what Google changed after the shared ancestor. leveldb in this release is commit `13da2d67` (11 August 2026). Its shared ancestor with Google's main is `0c40829` (5 November 2019). Since that ancestor the fork changed 32 files, **+242** **−194**. Google changed 115 files, **+2,335** **−1,886**. Set side by side at this release, 114 files differ: 1,973 lines are only in the fork and 2,374 are only in Google. Most of the fork's own patch is `util/env_windows.cc` (**+88** **−39**). crc32c is commit `efb8ea04` (30 August 2024), ancestor `21fc8ef` (8 February 2022). The fork changed 15 files, **+46** **−24**, mostly `src/crc32c_arm64.cc` (**+22** **−12**) and the dropped `third_party` submodules. Google changed 8 files, **+54** **−38**. The

direct gap is 17 files: 68 lines only in the fork, 62 only in Google. These counts are git numstat, so they include comments and blanks. Google's leveldb tip for the comparison is `bb74ef73` (8 October 2026). Google's crc32c tip is `2bbb3be` (7 April 2025).





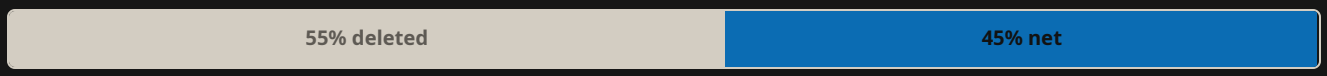
Fetched at build time, and absent from the checkout, so they have no line count here: Qt 6.8.4, Boost 1.91.0, SQLite 3.50.4, ZeroMQ 4.3.5, Cap'n Proto 1.5.0, qrencode 4.1.1, systemtap 5.3. Those are wider-community libraries. `src/bench/nanobench.h` is 2,388 lines of third-party nanobench sitting inside the test tree.

Public-key checks in `src/pubkey.cpp` call into secp256k1. The signature math is a Core project used by other software. It is not part of the node's own source layout, and it is larger than the first-party rules.

VIII. Who added the lines

1,122,543 lines were added and **614,627** were deleted. **55%** of what was added was later deleted. Net is added minus deleted. It is not the lines still in the tree. That count is section IX. The filter is code files only. Comments and blanks count. Vendored trees, Qt translations, and build scripts do not.

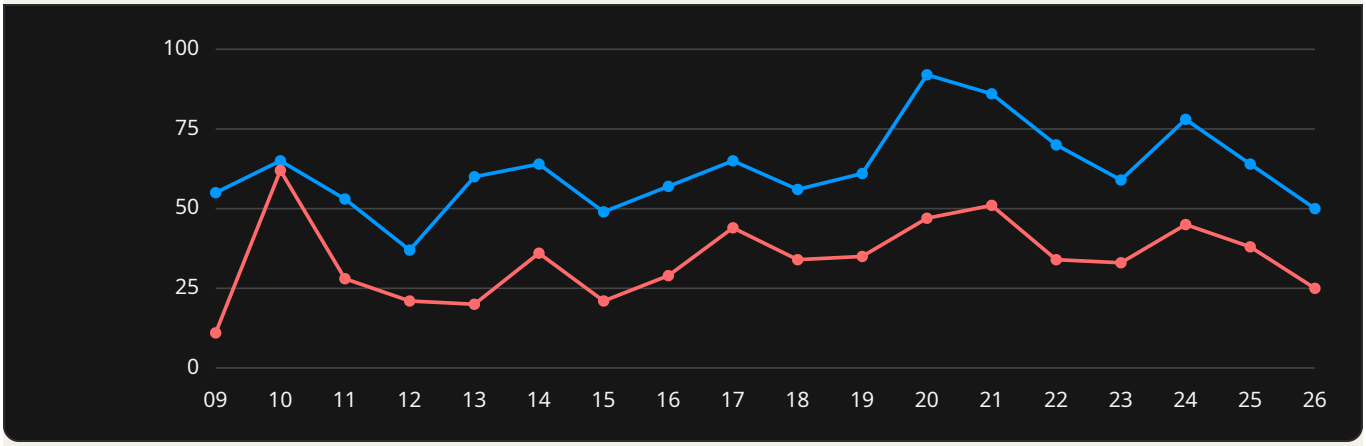
Of the lines added to code files, through 26 October 2026



Added **+1,122,543** Deleted **−614,627** Net **+507,916**

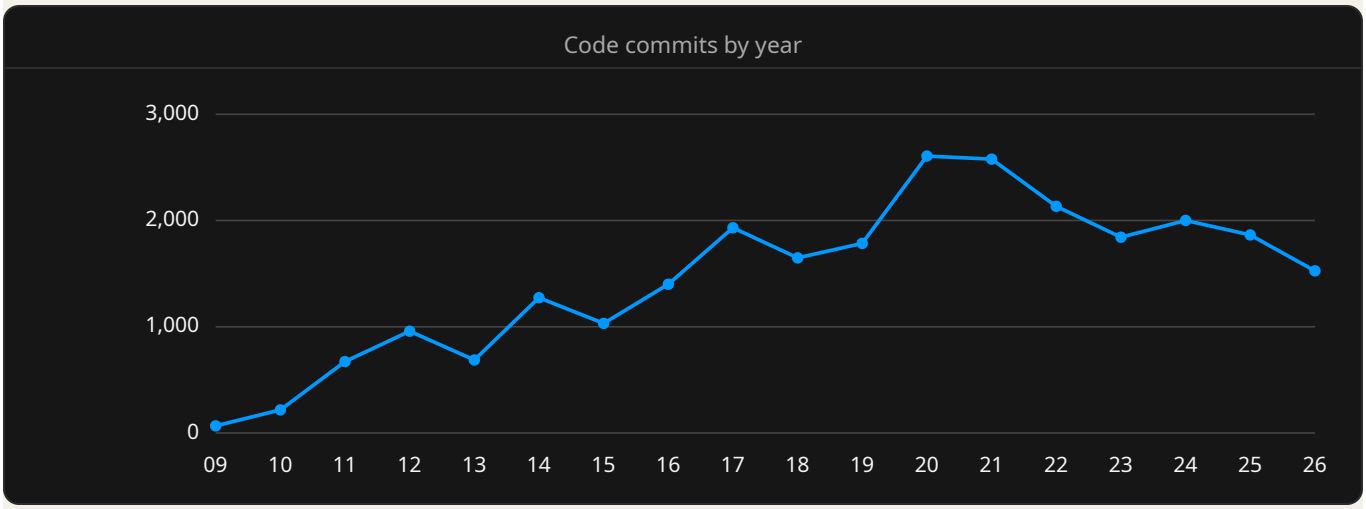
33,670 non-merge commits through this release. 26,226 of them touched the filtered code files. The other 7,444 touched only build, docs, translations, or vendored paths.

Lines added and deleted, by year

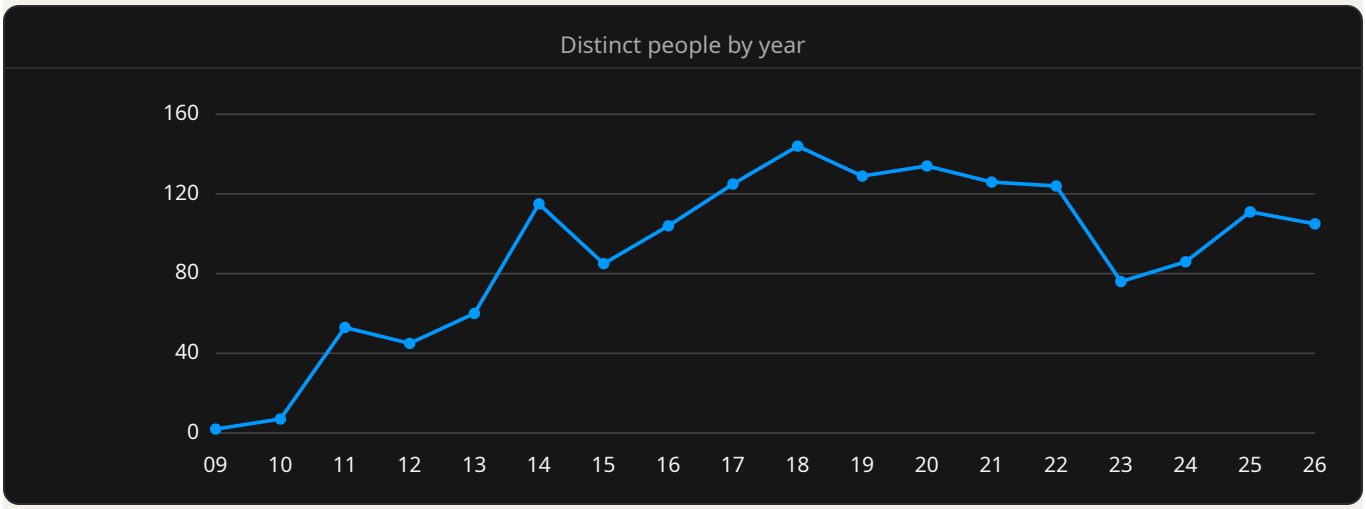


Thousands of lines, rounded. 2020 added 91,782 and deleted 47,157. 2010 added 65,289 and deleted 62,104, a net of 3,185. The largest net is 2020, at 44,625. 2009's net is 44,158. 2026 runs through 26 October.

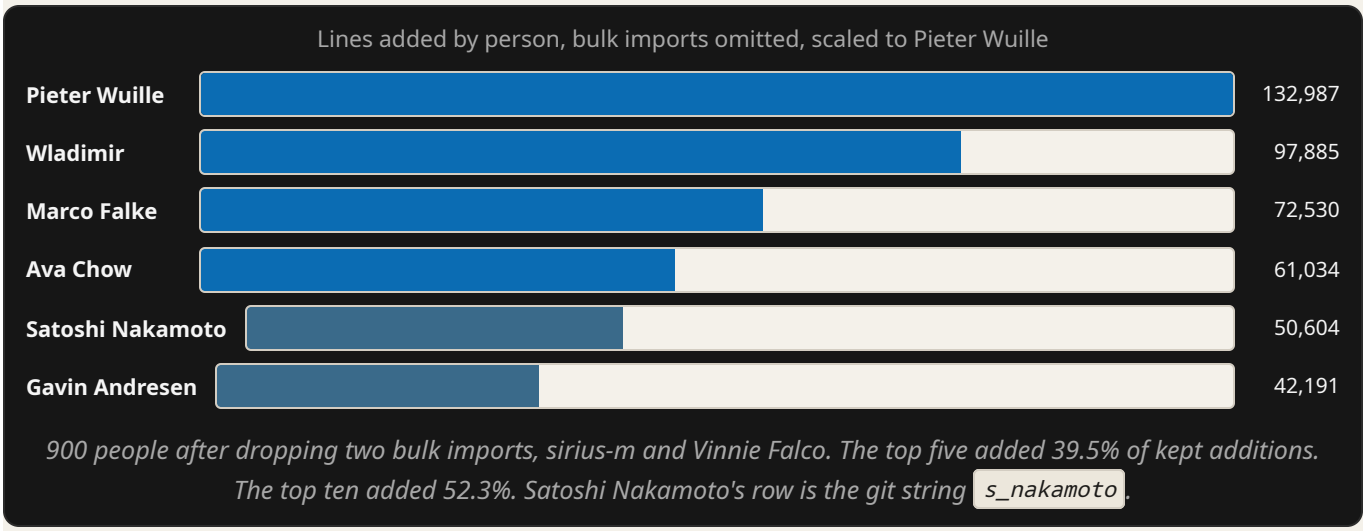
Code commits by year



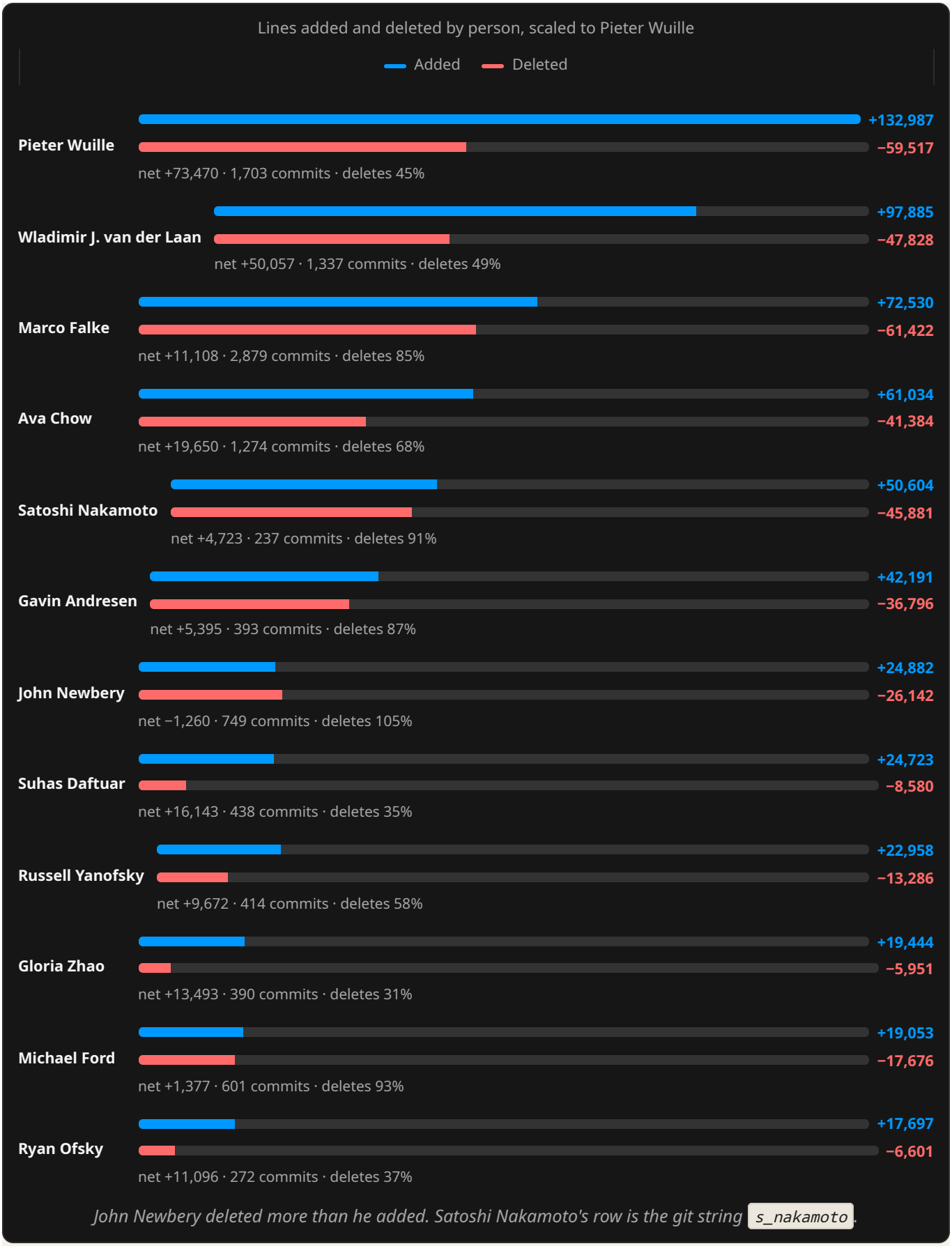
The peak is 2,606 commits in 2020. 2026, through 26 October, is 1,526.



Two git names that share an email are one person, and so is one name that used two addresses. The peak is 144 people in 2018. 2023 falls to 76. 2026, through 26 October, is 105.



900 people after dropping two bulk imports, sirius-m and Vinnie Falco. The top five added 39.5% of kept additions. The top ten added 52.3%. Satoshi Nakamoto's row is the git string `s_nakamoto`.



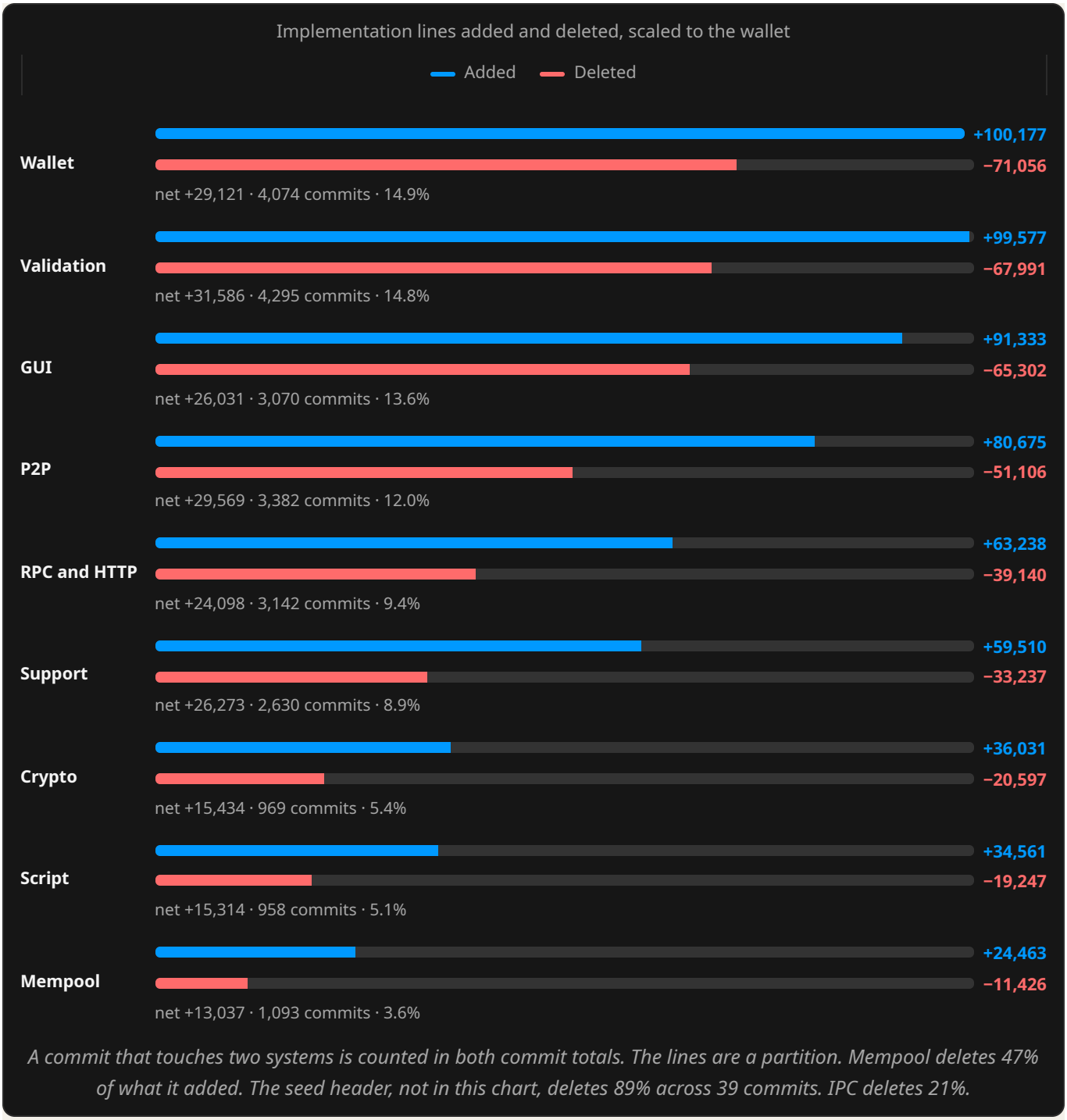
John Newbery deleted more than he added. Satoshi Nakamoto's row is the git string `s_nakamoto`.

Michael Ford's code-file net is 1,377 because this filter skips build scripts. His merge share is a different measurement, in *Who Controls Bitcoin*. Line authorship and merge authority are not the same ranking. Pieter Wuille leads additions. The merge log does not.

On a coarser directory sort of the same git lines, tests gained the most net (167,063 of 302,025 added). Validation kept 31,808 of 110,917. The GUI kept 23,077 of 84,779. Crypto kept 8,120 of 27,526. That sort counts all of `src/node` as P2P, so it does not match the 16,880 current P2P bar. The next section uses the same file lists as the bars in section III.

Where the work went

The 55% figure above counts every filtered code file, including tests. Restricted to implementation files, and split with the same system lists as section III, **672,059** lines were added and **431,312** were deleted. **64.2%** of what was added was later deleted. No system has a majority. Wallet is the largest share of lines added, at **14.9%**. Validation is **14.8%**. The top four, wallet, validation, the GUI, and P2P, are **55.3%**.



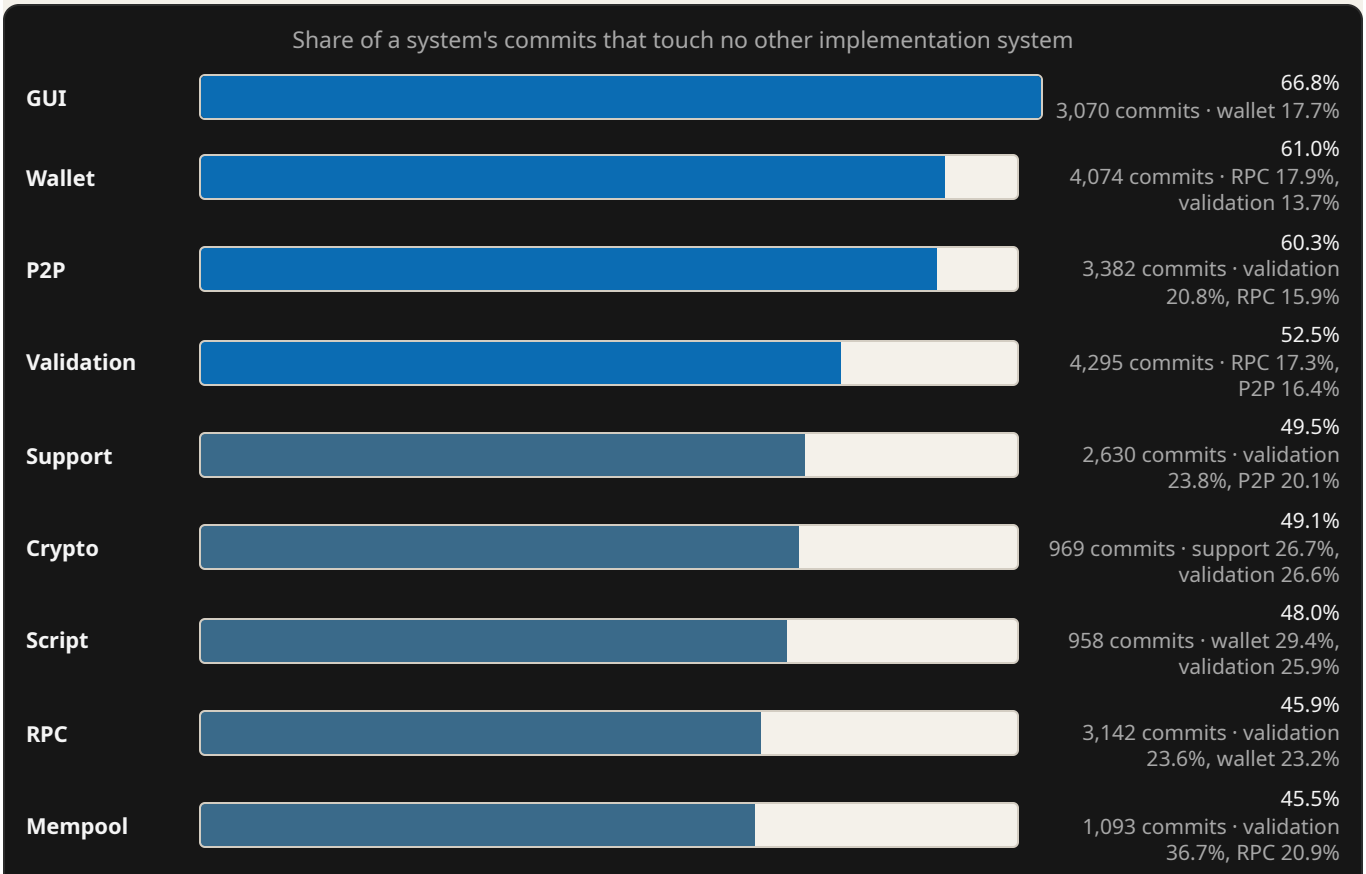
A commit that touches two systems is counted in both commit totals. The lines are a partition. Mempool deletes 47% of what it added. The seed header, not in this chart, deletes 89% across 39 commits. IPC deletes 21%.

The early years are the GUI. In 2009–2013 the GUI is **28.7%** of implementation lines added. In 2022–2026, through 26 October, validation leads, at **16.2%** (25,716 of 158,548).

The nets in that later era are validation +11,263, mempool +7,504, P2P +7,339, support +6,916, RPC +5,816, wallet +2,061, and the GUI +261. Wallet is the largest tree today because older lines survived. It is not where the recent lines went.

What changes in the same commit

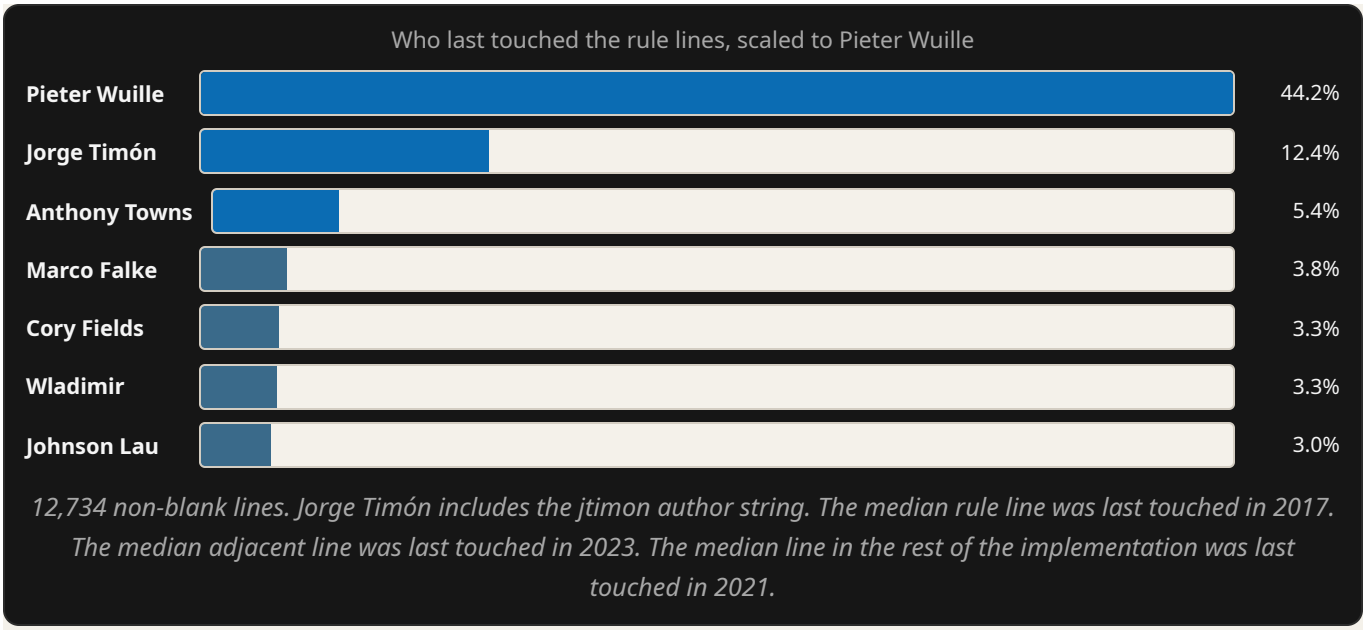
61.0% of wallet commits touch no other implementation system, and **13.7%** of them also touch validation. The tight pair is mempool and validation: **36.7%** of mempool commits also edit validation. RPC is the least separate. It is alone in **45.9%** of its commits, and those commits also touch validation **23.6%** of the time and wallet **23.2%**.

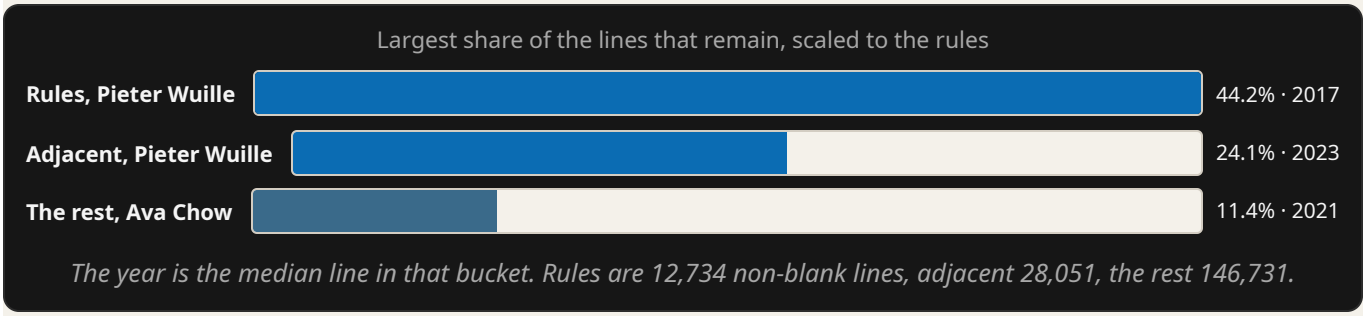


All history through 32.1. A commit that edits two systems counts once in each. Alone means no other classified implementation system. The last three years, 26 October 2023 through this release, are close for wallet (68% alone) and validation (62%). The GUI moved, from 66.8% alone over the whole history to 47% in that window.

IX. Who last touched what remains

`git blame -w` on the 756 implementation files. Each non-blank line goes to the last commit that changed it. `validation.cpp` is split on the same function boundaries as section IV. Comments count, so these totals sit above the tokei code lines: 12,734 non-blank rule lines against 10,122 code lines.



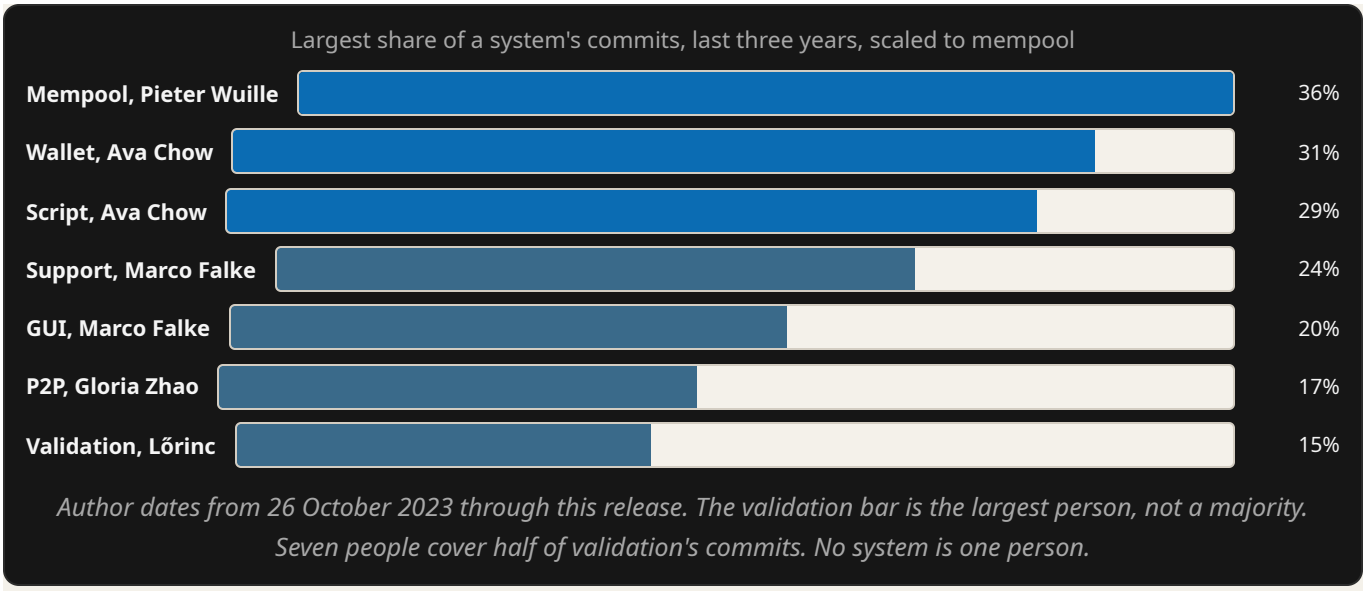


Across all 187,516 non-blank implementation lines, Pieter Wuille last touched 13.9%, Ava Chow 9.1%, Marco Falke 7.6%, and Wladimir J. van der Laan 5.9%. Those four, plus TheCharlatan, Hennadii Stepanov, Russell Yanofsky, and Vasil Dimov, last touched 49.0%. Gloria Zhao is third on the adjacent lines, at 6.9%, and tenth overall.

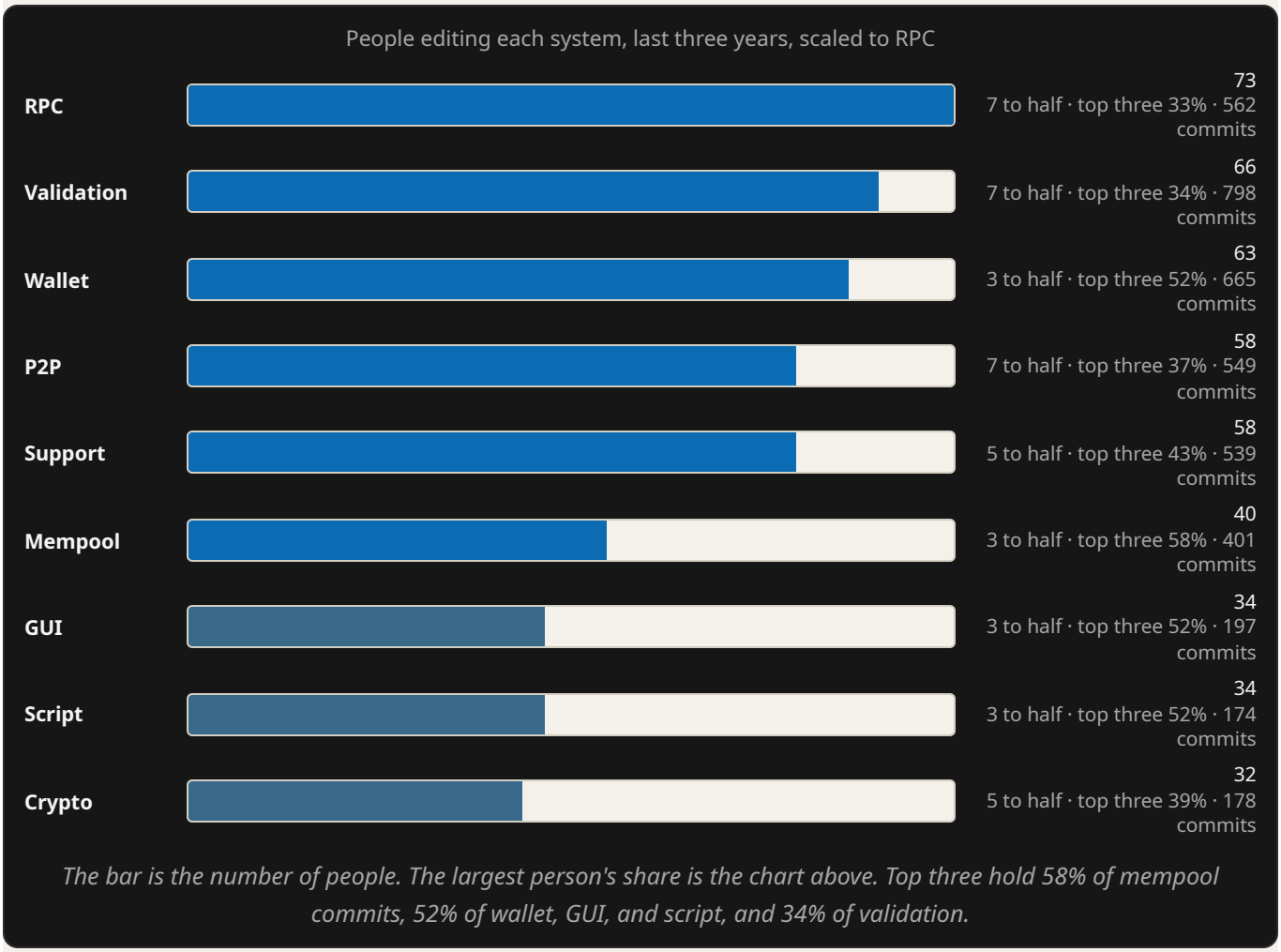
The rule lines are older than the code around them, and one person last touched 44% of them. The additions chart in section VIII is a different question. Satoshi's author string leads a large share of historical additions and is absent from the lines that remain. Who would make the next change is the next section.

Who still edits a system

Over the three years ending 26 October 2026, validation's 798 commits come from 66 people, and seven of them cover half. Lőrinc is 15%. Marco Falke is 10%. TheCharlatan, including commits signed sedited, is 9%. Mempool is the concentrated system: Pieter Wuille is 36% and Suhas Daftuar is 12%, just under half together, and three people cover half. Wallet takes three people to reach half. Ava Chow, including commits signed Andrew Chow, is 31% of them.



Author dates from 26 October 2023 through this release. The validation bar is the largest person, not a majority. Seven people cover half of validation's commits. No system is one person.

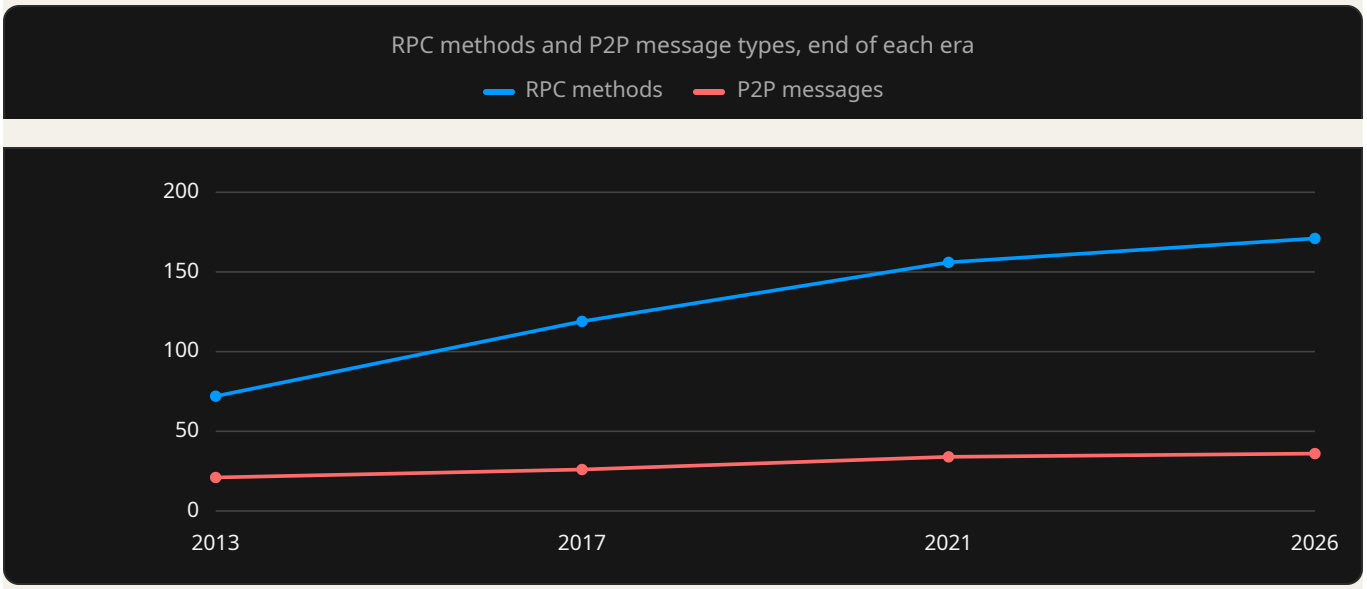


The bar is the number of people. The largest person's share is the chart above. Top three hold 58% of mempool commits, 52% of wallet, GUI, and script, and 34% of validation.

Two git names that share an email are one person. RPC has the most people, 73, and seven of them reach half of its commits. The GUI is the quiet tree, 197 commits. ismaelsadeeq is third on mempool, at 9%, after Pieter Wuille and Suhas Daftuar.

The size of the contract

At this release the tree exposes **171** RPC methods, **36** P2P message types, and **5** named deployment slots. RPC grew in every era and shed names in every era after 2013. The deployment list gained taproot by the end of 2021. In this release the named slots are bip34, bip65, bip66, csv, and segwit. The taproot rule is still in the script interpreter.



RPC by era: **+72 −0**, then **+52 −5**, **+53 −16**, **+29 −14**. P2P: **+21 −0**, then **+6 −1**, **+9 −1**, **+2 −0**. Named deployment slots went 1, 5, 6, 5.

The names behind those changes:

- By 2017, P2P added blocktxn, cmpctblock, feefilter, getblocktxn, sendcmpct, and sendheaders, and dropped alert.
- By 2021 it added addrv2, sendaddrv2, the compact-filter messages, and wtxidrelay, and dropped reject. This release added sendtxrcncl and feature.
- RPC dropped getgenerate, gethashespersec, getinfo, getwork, and setgenerate by 2017. By 2021 the account API was gone, along with addwitnessaddress, bumpfee, estimatefee, and signrawtransaction. This release dropped the legacy import and dump methods.
- The 2013 slot is the block-version-2 height-in-coinbase rule, later named bip34. By 2017 the slots were bip34, bip65, bip66, csv, and segwit. Taproot was the slot added by 2021. This release names the five buried deployments. The taproot rule is still in the interpreter.

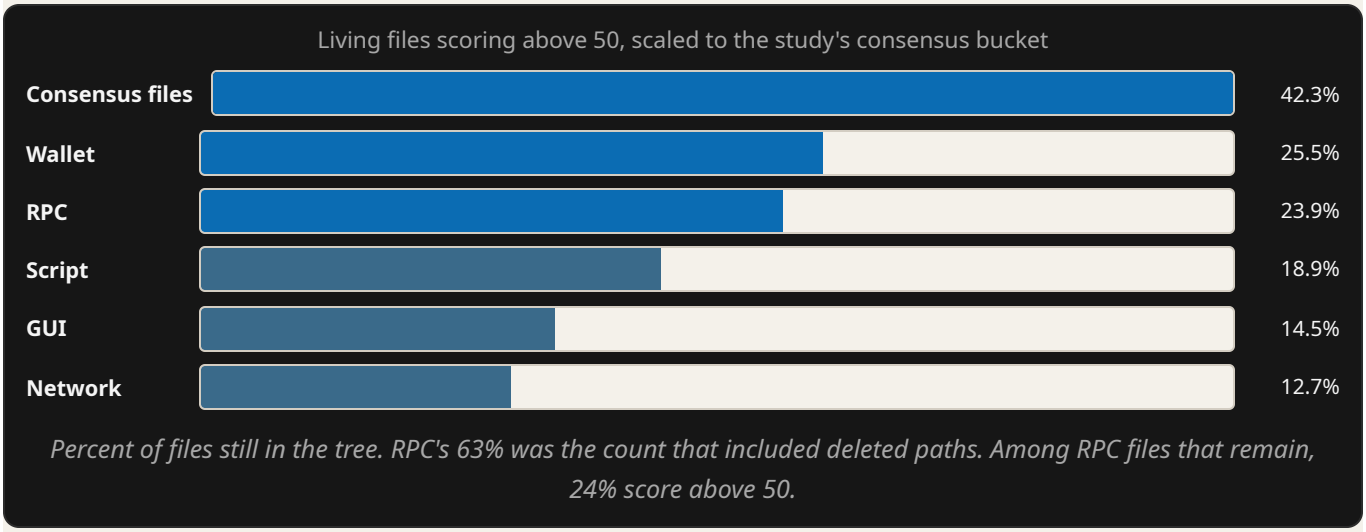
Commit-message counts of the words "bug fix" are not a defect rate. Cyclomatic complexity restates the files the line counts already flag as large. Neither is in this article.

X. Patch debt

The January 2026 debt study scores a file from how often it is edited in small diffs and how long it has gone without a large rewrite. This is the same formula, rerun on **17,072** merged pull requests from 3 January 2011 through 12 October 2026. It is a churn score. It is not a bug count. The pull-request dump ends before the 32.1 tag, so the scores do not see merges from 13 October through 26 October.

Counting every path that history ever touched, including files that are gone, **28.2%** score above 50 and the patch-to-refactor ratio is **3.0**. That reproduces the January headline (26.8% and 3.0). RPC is 63% of that wider count.

Restricted to files still in this tree, **15.8%** of files score above 50, and those files hold **10.2%** of current code lines. High-scoring files are smaller than average. Many of the old high scores were paths that no longer exist (`main.cpp`, `ui.cpp`). Among living files the ratio is **3.6** patches per refactor. Wallet is **6.9** to 1. The study's own consensus bucket, 26 files, is patched once per refactor, and 42% of those files still score above 50. That bucket is not the 10,122 rule lines.



The highest living source files on this score are `src/index/base.cpp` (80, 376 lines), `src/rpc/util.cpp` (79.4, 1,249 lines), `src/wallet/walletdb.h` (79.3), `src/interfaces/chain.h` (77.9), and `src/validationinterface.cpp` (72.3).

XI. What the count is for

The rules that have to be specified, audited, and matched by any other implementation are 10,122 code lines, plus a signature library that is larger than they are. The program operators download is the other 93% of the implementation, the tests, and the bundled trees. Treating a release as "the consensus rules" collapses those piles.

Don't Trust, Verify is about that collapse at the review layer. The first independent audit of Bitcoin Core, Quarkslab in September 2025, scoped P2P, mempool, block validation, and chain state. Those are mostly the adjacent lines in section V, plus some of the checks. The interpreter, the hash implementations, and the type of the transaction are the rest of the 10,122, and secp256k1 is another 76,503.

Who Controls Bitcoin measures who merges. This article measures who wrote lines and who last touched the lines that remain. The rankings differ. Merge concentration answers who can land a change. The blame in section IX answers whose text is still in the rule files. Both can be true, and neither is a substitute for the other. The later sections answer three further questions. Wallet commits also touch validation 13.7% of the time. Over the last three years, validation's commits come from 66 people. A release exposes 171 RPC methods, 36 P2P message types, and 5 deployment slots.

Bitcoin Core: The Biggest Fallacies argues the client is a monolith because the wallet, the network, and the rules ship together. The line count is the size of that claim. Seven percent is the rules. Eighty percent is the rest of the same binary.

What Bitcoin's Stalled Proposals Tell You uses the authorship Gini from the governance research. That Gini is on pull-request authorship. The added-line shares in section VIII are a third cut. Top-five line additions at 39.5% and a historical authorship Gini near 0.85 are the same shape at two resolutions. They are not the same number.

The cost of storing the chain this program validates is a separate account, in *Full Cost of Running a Bitcoin Node*.

XII. Method

Snapshot. Bitcoin Core **32.1**, branch `32.x`, commit `f0de87f040621468b956f01f526a43a010310501`, 26 October 2026, "Merge bitcoin/bitcoin#36658: [32.x] Finalise 32.1". The tag matches the branch tip. Not shallow. 50,657 commits, 16,987 of them merges. Line counts are this release. Authorship is history through this commit.

Code lines. tokei 14.0.0. A file is in one bucket. Implementation is `src/` excluding vendored prefixes (`secp256k1`, `leveldb`, `crc32c`, `minisketch`, `univalue`, `libmultiprocess`, `ctaes`), Qt locale, tests, fuzz, bench, markdown, and `CMakeLists.txt`. The system split uses explicit file lists for `src/node`, because that directory is not all P2P. `src/node/mining_args.cpp`, `mining_args.h`, and `mining_types.h` are in mining templates.

The three-way cut. Rule files are the SHA-256 and RIPEMD-160 sources, `hash.cpp` and `hash.h`, the script interpreter and script data, `pubkey`, `arith_uint256`, `uint256`, `src/primitives`, `src/consensus`, proof of work, versionbits, deployments, and signet. `validation.cpp` is split by

function start, comment-stripped, then scaled from 4,187 raw code-like lines to tokei's 4,247. Adjacent is chainstate and storage, policy and mempool, indexes, and mining templates, including the matching parts of `validation.cpp`. SHA-1, SHA-3, and SHA-512 stay in the rest. The three parts sum to 151,004.

Tests. The named rule unit and fuzz files are the earlier filename list (4,517) plus integer, hash, script, and block tests that name a rule type (1,403). Functional rule tests are an explicit basename list: `feature_block`, `feature_taproot`, `feature_segwit`, `feature_cltv`, `feature_csv_activation`, `feature_dersig`, `feature_nulldummy`, `feature_bip68_sequence`, `feature_signet`, `feature_versionbits_warning`, `p2p_segwit`, `p2p_invalid_block`, `p2p_invalid_tx`, `p2p_mutated_blocks`. Adjacent adds an explicit list of mining, assumeutxo, pruning, index, fee, package, and chainstate tests to the mempool and policy filename prefix. A test outside those lists stays in the rest. That undercounts a functional test that exercises the interpreter under another name.

History. `git log HEAD --no-merges --numstat`. Renames of the form `{old => new}` are rewritten to the new path. Identity merges every pair of git names that share an email, and one name that used two addresses, plus the published aliases: Wladimir's three spellings, Ava Chow and Andrew Chow, Marco Falke and MarcoFalke, Gloria Zhao and glozow, Michael Ford and fanquake, jtimon and Jorge Timón. The tables use the release-note names. Satoshi Nakamoto's row is the `s_nakamoto` string. People who share only a first name stay separate. Ryan Ofsky and Russell Yanofsky are different people.

Systems over time. The same numstat history, implementation paths only, classified with the section III file lists, including the explicit `src/node` split. A path is classified by the name it had in that commit. Vendored trees and tests are out. A commit that touches two systems counts in both commit totals. "Only this system" means the commit touched no other classified implementation system. The three-year window is author dates from 26 October 2023 through this release. The people column merges git names that share an email, and the same published aliases as the history.

The contract. RPC names are the command table when that table carries the method name (2013 and 2017) and `RPCHelpMan` / `RPCMethod` constructors when it does not (2021 and this release). Hidden methods are included. P2P names in 2013 are the message strings the network dispatcher in `src/main.cpp`, `src/net.cpp`, and `src/alert.cpp` receives or sends. From 2017 on they are the `NetMsgType` strings. Deployment slots are `DEPLOYMENT_*` enumerators other than testdummy, plus the block-version-2 height rule, with height-field aliases of the same fork collapsed to one name. Era tips are the last commit on this history at the end of 2013 (`aec55a07`), 2017 (`c9439e73`), 2021 (`1831d43e`), and this release.

Blame. `git blame -w --line-porcelain` on each implementation file. Whitespace-only changes do not move the line. Blank lines are dropped. The same alias list as the history.

Debt. `scripts/analysis/code_turnover_analysis.py` in the Bitcoin Governance Research repository, run on `data/github/prs_raw.jsonl` (25,506 pull requests, 17,072 merged with a date, last merge 12 October 2026). The score weights patch frequency, time since a large rewrite, age of files the script labels consensus, and modifications per estimated line. Change type is a size heuristic: a refactor is a change over 30% of the estimated file or over 200 lines. The script's subsystem labels are filename patterns. They are not the three-way cut in section IV. Living-file membership is the 32.1 tree.

Sources

- Bitcoin Core 32.1, commit `f0de87f040621468b956f01f526a43a010310501`, branch `32.x`, 26 October 2026
- tokei 14.0.0, file-level report on that tree
- Bitcoin Governance Research, technical debt analysis, formula rerun 30 October 2026 on the pull-request dump through 12 October 2026
- *Who Controls Bitcoin*, *The Vertical Layer Problem*, *Why Bitcoin Needs a Specification That Humans Can Read*, *Don't Trust, Verify*, *Bitcoin Core: The Biggest Fallacies*, *What Bitcoin's Stalled Proposals Tell You*, *Full Cost of Running a Bitcoin Node*

SECURESOVEREIGN

Markdown is the canonical source: https://secsov.com/articles/the-rules-are-7-percent/index.md

Formatted snapshot of https://secsov.com/articles/the-rules-are-7-percent

Josh · Secure Sovereign